Tokenary (Project Y) — Executive Summary for Funders (Hybrid: research + engineering + AI compute)

What it is: Tokenary is a structured, field-preserving token system that converts dictionary entries and linguistic concepts into a stable JSON schema with explicit *null* values. It is designed to make AI-assisted interpretation and research **repeatable**, **auditable**, and **far less error-prone** than ad■hoc prompting or one-off web research.

Why it matters: Current AI workflows are expensive because they repeatedly re-derive the same meanings from scratch, and they hallucinate when ambiguity is high. Tokenary reduces waste by anchoring meaning to stable tokens and reusing validated structure over time.

Funding request (what money buys):

A hybrid package covering: (1) engineering to harden parsers and schema guarantees; (2) research to design enrichment/verification methods; (3) controlled AI compute to accelerate enrichment for high-value subsets without sacrificing correctness.

System overview (two-cycle pipeline):

GCIDE (local)	→	Cycle 1: Parse headings	→	A–Z JSON tokens (schema preserved)
Wiktionary (local DB)	→	Cycle 2: Enrich (POS/labels/taxon hints)	→	Verification + audit trail

The problem

AI is powerful but operationally inefficient for sustained projects. In practice, long-running efforts hit three repeating barriers: **(a) ambiguity** (the model “guesses” rather than anchors), **(b) rework** (the same meaning is re-derived again and again), and **(c) schema drift** (tools silently change field shapes, breaking downstream code). Tokenary is a direct response: preserve a strict schema; capture meanings as reusable tokens; and record uncertainty explicitly via nulls.

What Tokenary delivers

1) Field preservation: every token shares one schema; blank fields are *null* (not omitted), so enrichment can be measured and audited.
2) Reuse: once a token is enriched and verified, future tasks can reuse it without repeated research.
3) Lower hallucination risk: constrained structure reduces free-form invention; verification is localized and repeatable.
4) Interpretation gains: separating function-words and lexical meaning improves signal-to-noise in language analysis (core to Project Y).

Progress to date (working prototype)

• **GCIDE ingestion:** exported **96,008** tokens from a local GCIDE text source into A–Z JSON files.
• **Schema discipline:** tokens retain a fixed JSON body with explicit nulls for future data entry.
• **Local enrichment base:** built a local **English Wiktionary SQLite** database (~1.34M entries kept) and joined categories into the Tokenary JSON.
• **Known failure modes documented:** (i) confusing headings/notes as headwords, (ii) POS/labels not mapping cleanly to ontology fields, (iii) naive taxonomy backfill yields low hit-rates, (iv) categories can be wrong if labels are interpreted as topic tags instead of word class.

Why this still needs funding

This is not a small script problem — it is a **data infrastructure build**. To reach “fundable impact,” Tokenary needs: (a) robust parsing (correct headword boundaries, handle notes, homographs, etymology blocks); (b) a trustworthy enrichment strategy (local-first, auditable, reversible); (c) a verification workflow (confidence scoring + sampling + human review for high-impact domains). Funding converts a working prototype into a reliable, repeatable pipeline.

Efficiency & impact claims (what improves, and why)

Tokenary aims to cut cost across four components of AI work: chat turns, reasoning effort, research overhead, and verification time. Expected outcomes once the pipeline is hardened:

Dimension	Baseline (common workflow)	Tokenary effect	Practical consequence
Chat efficiency	Repeated clarification loops	Stable token lookup + constraints	Fewer turns; faster completion
Thinking cost	Ambiguity drives long reasoning	Nulls + schema reduce inference	Lower token usage; fewer contradictions
Research effort	Repeated web searches	Reuse validated structured facts	Less searching; more building
Correctness / verification	Manual spot-checking each time	Check once; reuse with audit trail	Lower error rate in repeat domains

Work plan (90-day fundable deliverables)

Engineering (weeks 1–6)
• Harden GCIDE parser: correct entry boundaries, ignore “Note:” as headwords, normalize casing/spacing, preserve unique IDs per sense.
• Enforce schema: validation tool that rejects any drift (missing fields, wrong types).
• Build enrichment hooks: local Wiktionary join (POS/labels), plus optional targeted sources per domain (biology/chemistry/etc.).

Research + verification (weeks 4–10)
• Define mapping rules: POS/labels → Tokenary category; label taxonomy hints → structured taxonomy fields with confidence scores.
• Sampling audit: measure correctness via random samples and error taxonomy; publish metrics.

AI compute (weeks 6–12)
• Use controlled LLM enrichment only where schema + sources constrain output (e.g., animals/plants/chemicals).
• Log citations/sources per filled field for traceability.

Budget framing (what to fund)

A typical hybrid micro-grant can support: **(1) developer time** (parser hardening, validation, pipeline); **(2) research time** (mapping rules, evaluation); **(3) compute** (targeted enrichment runs) plus storage/CI. Tokenary is designed to compound: every verified token reduces future cost.

Who benefits

• Researchers and digital-humanities teams needing reproducible language resources.
• Fact-checking / online safety projects needing stable semantic primitives.
• Tooling for accessibility: reducing cognitive load via structured “meaning anchors.”
• Open-source ecosystems: reusable dataset/pipeline rather than proprietary black-box enrichment.

Contact / next step

Seeking funders who support open-source digital public goods, trustworthy/safer internet infrastructure, and applied language technology. Pilot funding enables a 90-day milestone: parser hardening + verified enrichment pipeline + published metrics.